Partial Failure of LLM Components in Distributed Recommender Pipelines

Introduction

Modern recommender systems are increasingly deployed as distributed, multi-stage pipelines with independent microservices being responsible for candidate generation, scoring and reranking [4, 18]. Recently, a trend of integrating Large Language Models into said pipelines has emerged, be it for the purposes of debiasing [5], zero-shot recommendations [10] or deeper understanding of the user [11]. This intersection creates the following dependability challenge - *How robust are distributed recommender system pipelines to partial failure of their LLM components and what detection, containment, and recovery mechanisms have been proposed or adapted for this setting?*

This review synthesizes literature from foundation-level dependability papers, distributed systems and LLM-augmented recommendation spheres. It examines how architectural placement of LLM determines the nature of failure, characterises the failure modes, and surveys mechanisms for detection, containment and recovery. A primary takeaway is that existing solutions either address infrastructure-level or semantic-level failures, but very few integrated frameworks simultaneously bridge the two layers.

LLM placement in the Recommender Pipeline

Most modern distributed recommender systems break their task down into cloud, edge and terminal nodes [4], similar to microservice architectural patterns, which offers independent deployment, but at the same time introduces cascading failure risk and increased resource demands [18]. Modern production architectures vary from hybrid solutions combining cached offline outputs with real-time inference [16] and hierarchical parameter servers with tiered storage systems [20] to fully disaggregated recommenders [7]. Huang et al. [7] explicitly notes enhanced failure isolation as an advantage of the latter, but their reasoning is solely architectural with no fault injection experiments validating it. Furthermore, the paper does not make an effort to analyse new network-level faults that RDMA (Remote Direct Memory Access) nodes introduce, all while Zhang [25] highlights such inter-node interactions as the main reason for information loss.

The pipeline stage that LLMs reside in decides which failure modes the system will face and what fallback options will be available.

At one end of this spectrum, LLMs may be utilised at runtime to rerank candidates with the objective of solving bias problems [5] and cold-start recommendation [10], or perform tasks like query rewriting and retrieval [26]. Because the LLM occupies a single stage, the remaining pipeline can continue operating if it degrades, providing a natural fallback boundary.

Removing LLM from serving, the LLM-Enhanced Recommender Systems (LLMERS) paradigm suggests integrating LLM into the training stage exclusively [11, 24], which eliminates runtime failures by design. Yet, neither Liu et al. [11] nor Zhang et al. [24] investigate the possibility of LLM producing faulty outputs during offline generation, perpetuating systematic errors in the form of model weights that are difficult to detect later.

At the opposite extreme, a unified approach merges retrieval and ranking into a monolithic model, eliminating interstage information loss at the cost of sacrificing fault isolation boundaries that allow handling partial failure [25]. Zhang et al. [25], however, do not comment on this dependability tradeoff, focusing exclusively on the approach's accuracy improvement. A fundamental tension can be traced: tighter integration improves recommendation quality, but progressively removes fault isolation boundaries for containment and recovery.

Placement differences have latency consequences. Singh et al. [16] describe latency increasing to 1.5s after adding a recommendation model to a retail system "making it unusable", although their evaluation uses a single 16-core VM and does not demonstrate how horizontal scaling would affect the figures. Nevertheless, this effect is potentially deal-breaking, considering how LLMs' inference is substantially slow for many real applications' standards [5, 11].

Failure modes of LLM components

Having established their positions in the pipeline, the next question is what failure modes LLM components introduce. This distinction is mostly defined by the semantic invisibility - an LLM can return a syntactically valid response while being semantically wrong - placing such failures outside of conventional microservice anomaly detectors' capabilities [17].

Hallucination occurs when a language model generates content that is plausible but factually wrong, like recommending items that are absent from the catalogue. Unlike a crash or a timeout, a hallucinated output reaches the user silently and degrades their experience and undermines trust without triggering system warnings. Jiang et al. [9] evaluate seven LLMs and find that hallucination rate is model-specific, making the selection itself a dependability question. While this is one of the most systematic evaluations of hallucinations in recommendation context, it has been conducted offline, which leaves out whether the hallucination rates hold under production conditions with dynamic catalogue.

Carraro and Bridge [5] observe the same issue in their diversity reranking experiments. In this context, invalid item IDs are replaced with random ones, directly degrading the recommendation quality. Zhu et al. [26] further warn that LLM-generated content entering the retrieval architecture can create feedback loops amplifying errors across stages, though this cascade mechanism is described hypothetically rather than demonstrated.

Non-determinism refers to an LLM property of producing different outputs over various iterations featuring identical inputs. In a pipeline, this means that retrying a failed request - a standard recovery strategy in distributed systems - may silently change the recommendation rather than reproducing the original result, reducing the user's trust and making debugging impossible.

Ouyang et al. [12] provide extensive evidence, with over 50% of their code generation tasks yielding 0 semantically equal outputs across 5 requests. Their claim that setting the temperature to 0 does not guarantee determinism is independently confirmed by Astekin et al. [2] in log parsing, establishing this as a cross-domain fault rather than a task-specific one.

Latency instability differs from standard slowness since LLM inference latency is unpredictable on a per-request basis. Jain et al. [8] demonstrate that mixing prefill and decode workload phases on the same GPU instance causes execution-time spikes despite all the instances being healthy. This makes breaching Service Level Agreement boundaries possible even with individual requests. Their workload-mixing model is a valuable formalisation, though the evaluation uses only four instances and does not assess behaviour at production scale.

Gray failure ties these modes together, describing technically operational components producing degraded outputs [15, 22]. Saberian and Basilico [15] describe production scenarios at Netflix where "all of the code is working flawlessly" yet recommendations are suboptimal. At the infrastructure level, Xiong et al. [22] demonstrate that hardware redundancies in GPU clusters mask gradual degradation, with mean time between incidents of 17.5 hours in Azure A100 clusters. A degraded GPU producing subtly corrupted LLM outputs that manage to pass health checks is a representation of infrastructure and semantic gray failure intersection that existing detectors do not address.

The resilience taxonomy of Berger et al. [3] provides foundational vocabulary for classifying these modes but was developed for IoT and does not account for semantic failure dimensions. Soldani and Brogi [17] survey microservice anomaly detection comprehensively and explicitly identify circuit breakers as countermeasures, yet an LLM returning hallucinated recommendations on time is invisible to the techniques they survey.

Comparing these failure modes reveals an asymmetry: Hallucination has a strong user impact, undermining their trust silently, but is the most detectable in principle, since the outputs could be validated against a known catalogue. Undermining retry-based recovery, non-determinism has the broadest scope - it complicates debugging for every pipeline stage with LLM integration. Latency instability is both conventionally observable and difficult to attribute to a root cause when phases are mixed. Gray failure combines the silent impact of hallucinations with infrastructure-level invisibility of healthy nodes, making detectability and recovery challenging. This asymmetry makes it difficult to address all four modes with a single strategy when examining existing frameworks.

Detect, contain, recover

Given said asymmetry, existing mechanisms address the “detect-contain-recover” task partially, but operate on different layers in architectural sense, hence the lack of an exhaustive universal solution.

Detection splits between two levels. At the infrastructure layer, solutions may include observability systems like eInfer [6] that reconstruct full execution timelines with minuscule overhead, complemented with validation systems that benchmark GPU health before incidents occur [22]. Together, they offer a strong detection mechanism, yet neither detects semantic faults, which other approaches target. Ross et al. [14] utilise RARR for detecting hallucinations, but admit that its agreement module produces excessive false positives, which, in recommender pipelines, would trigger unnecessary fallbacks, degrading output when the LLM was correct. This is acknowledged, but no modifications are suggested. Yang et al. [23] use metamorphic relations to detect hallucinations, substantially increasing the token cost - which may hinder practical implementation. Sun et al. [19] mitigate hallucinations via decoding at generation stage with effective overhead, but cannot catch errors from degraded inputs. Among the three, none addresses non-determinism or latency instability, leaving two of the four failure modes without any semantic detection mechanism. Overall, these approaches have to compromise between serving cost and detection thoroughness.

Containment limits failure propagation. Phase-aware workload balancing [8] distributes LLM requests to reduce latency, but optimises for performance, not correctness, and cannot route around an instance responding quickly with hallucinated responses. Constraining LLM outputs with guardrails is another mechanism that pre-defines valid responses, preventing hallucinations from propagating downstream, as demonstrated by Pal Chowdhury et al. [13] in educational context. Using a formal model that maps LLM outputs to a fixed set of permitted states in recommender context would mean item grounding by rejecting outputs that reference non-existent items [9]. At the infrastructure level, cooperative degradation frameworks like Phoenix [1] use criticality tags for selective container turn-off - handling 100,000-node failures in 10 seconds. In a recommender pipeline retrieval is arguably more essential than LLM reranking - still, such approaches assume classical hard failure and cannot act on containers that are producing hallucinated outputs yet running, which is precisely the failure mode that distinguishes LLMs from conventional microservices. Each containment mechanism maps to a narrow failure class with no single strategy spanning all four failure modes identified in the previous section.

Recovery restores availability after failure. Two strategies emerge: heterogeneous model and cache-based serving. For the former, Wu et al. [21] maintain smaller model variants as backup replicas and progressively upgrade from the fastest fallback to the most capable as resources allow. In a recommender pipeline that would mean serving traditionally ranked items on LLM failure and incrementally restoring its function afterwards. However, in this realm smaller models may feature worse position bias [9] or hallucinate [12] more frequently, not merely reducing accuracy, but shifting the failure profile entirely. Cache-based recovery [20] offers a complementary path: hierarchical storage tiers already present in production recommenders form a natural degradation chain. Pre-generated LLM outputs [11, 24] could add an LLM-failure-tolerant layer, but how quickly cached recommendations become outdated remains unstudied. Taken together, neither strategy includes a mechanism for detecting whether the cached or recovered output is semantically degraded, leaving hallucination and non-determinism unaddressed.

Conclusion

With recommender platforms aiming to increase personalisation to win user attention, it is reasonable to expect increased LLM presence in these systems in the near future. Recommender researchers are already documenting what fails when LLM enters the pipeline [5, 9, 12]. The distributed systems community has developed mechanisms for infrastructure failure detection [6], and containment and recovery [1, 21, 20].

Surprisingly, there is a lack of studies that bridge these into an integrated dependability framework. While there are solutions that make recommenders robust to partial failure, very few are practically adaptable to the LLM failure modes without severe compromises. Infrastructure monitoring cannot effectively detect semantic failures for cost [23] or precision [14] reasons - tradeoffs that are not always acceptable in real-world applications; while recovery mechanisms assume failure modes that do not include grey degradation [1]. The fundamental challenge is that the most dangerous failure modes for the users - hallucinations and gray failures - are those least visible to the infrastructure, creating a blind spot neither community has closed.

Future research should prioritise empirical studies that experiment with LLM stability under stress, multi-tier fallback architectures designed to coherently act on grey failures, and benchmarks measuring pipeline-level robustness under LLM component degradation. Ideally, proposed solutions should be applicable to real-world production systems.

References

1. Agrawal, K., & Abdu Jyothi, S. (2025). Cooperative Graceful Degradation in Containerized Clouds. *International Conference on Architectural Support for Programming Languages and Operating Systems - ASPLOS*, *1*, 214–232. https://doi.org/10.1145/3669940.3707244
2. Astekin, M., Hort, M., & Moonen, L. (2024). An Exploratory Study on How Non-Determinism in Large Language Models Affects Log Parsing. *Proceedings - 2024 IEEE/ACM 2nd International Workshop on Interpretability, Robustness, and Benchmarking in Neural Software Engineering, InteNSE 2024*, 13–18. https://doi.org/10.1145/3643661.3643952
3. Berger, C., Eichhammer, P., Reiser, H. P., Domaschka, J., Hauck, F. J., & Habiger, G. (2022). A Survey on Resilience in the IoT: Taxonomy, Classification, and Discussion of Resilience Mechanisms. *ACM Computing Surveys*, *54*(7). https://doi.org/10.1145/3462513
4. Cai, Q., Cao, J., Xu, G., & Zhu, N. (2024). Distributed Recommendation Systems: Survey and Research Directions. *ACM Transactions on Information Systems*, *43*(1). https://doi.org/10.1145/3694783
5. Carraro, D., & Bridge, D. (2026). Enhancing Recommendation Diversity by Re-ranking with Large Language Models. *ACM Transactions on Recommender Systems*, *4*(2), 1–40. https://doi.org/10.1145/3700604
6. Chu, K., Su, J., Zhang, Y., Zhao, C., Yang, Y., Zheng, Y., Lin, S., Zhao, S., & Zhang, W. (2025). EInfer: Unlocking Fine-Grained Tracing for Distributed LLM Inference with eBPF. *eBPF 2025 - Proceedings of the 2025 3rd Workshop on eBPF and Kernel Extensions, Part of SIGCOMM 2025*, 76–83. https://doi.org/10.1145/3748355.3748372
7. Huang, Y., Qiu, Y., Yang, Z., Dai, Y., Wu, D., Lai, F., Xing, J., & Chen, A. (2025). *Towards Fully Disaggregated Recommendation Model Serving*. 38–45. https://doi.org/10.1145/3725783.3764389
8. Jain, K., Parayil, A., Mallick, A., Choukse, E., Qin, X., Zhang, J., Goiri, Í., Wang, R., Bansal, C., Rühle, V., Kulkarni, A., Kofsky, S., & Rajmohan, S. (2025). Performance Aware LLM Load Balancer for Mixed Workloads. *EuroMLSys 2025 - Proceedings of the 2025 5th Workshop on Machine Learning and Systems*, 19–30. https://doi.org/10.1145/3721146.3721947
9. Jiang, C., Wang, J., Ma, W., Clarke, C. L. A., Wang, S., Wu, C., & Zhang, M. (2025). Beyond Utility: Evaluating LLM as Recommender. *WWW 2025 - Proceedings of the ACM Web Conference*, 3850–3862. https://doi.org/10.1145/3696410.3714759
10. Kieu, H. D., Nguyen, M. D., Nguyen, T. S., & Le, D. D. (2025). Keyword-driven Retrieval-Augmented Large Language Models for Cold-start User Recommendations. *WWW Companion 2025 - Companion Proceedings of the ACM Web Conference 2025*, 2717–2721. https://doi.org/10.1145/3701716.3717855
11. Liu, Q., Zhao, X., Wang, Y., Wang, Y., Zhang, Z., Sun, Y., Li, X., Wang, M., Jia, P., Chen, C., Huang, W., & Tian, F. (2025). Large Language Model Enhanced Recommender Systems: Methods, Applications and Trends. *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, *2*, 6096–6106. https://doi.org/10.1145/3711896.3736553
12. Ouyang, S., Zhang, J. M., Harman, M., & Wang, M. (2025). An Empirical Study of the Non-Determinism of ChatGPT in Code Generation. *ACM Transactions on Software Engineering and Methodology*, *34*(2). https://doi.org/10.1145/3697010
13. Pal Chowdhury, S., Zouhar, V., & Sachan, M. (2024). AutoTutor meets Large Language Models: A Language Model Tutor with Rich Pedagogy and Guardrails. *L@S 2024 - Proceedings of the 11th ACM Conference on Learning @ Scale*, 5–15. https://doi.org/10.1145/3657604.3662041
14. Ross, J. J., Khramtsova, E., van der Vegt, A., Koopman, B., & Zuccon, G. (2025). RARR Unraveled: Component-Level Insights into Hallucination Detection and Mitigation. *SIGIR 2025 - Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 3286–3295. https://doi.org/10.1145/3726302.3730337
15. Saberian, M., & Basilico, J. (2021). RecSysOps: Best practices for operating a large-scale recommender system. *RecSys 2021 - 15th ACM Conference on Recommender Systems*, 590–591. https://doi.org/10.1145/3460231.3474620
16. Singh, R. K., Mishra, M., & Singhal, R. (2023). *Scalable High-Performance Architecture for Evolving Recommender System*. 154–162. https://doi.org/10.1145/3578356.3592594
17. Soldani, J., & Brogi, A. (2022). Anomaly Detection and Failure Root Cause Analysis in (Micro) Service-Based Cloud Applications: A Survey. *ACM Computing Surveys*, *55*(3). https://doi.org/10.1145/3501297
18. Soldani, J., Tamburri, D. A., & van den Heuvel, W. J. (2018). The pains and gains of microservices: A Systematic grey literature review. *Journal of Systems and Software*, *146*, 215–232. https://doi.org/10.1016/j.jss.2018.09.082
19. Sun, Z., Si, Z., Zang, X., Zheng, K., Song, Y., Zhang, X., & Xu, J. (2025). LargePiG for Hallucination-Free Query Generation: Your Large Language Model is Secretly a Pointer Generator. *WWW 2025 - Proceedings of the ACM Web Conference*, 4766–4779. https://doi.org/10.1145/3696410.3714800
20. Wei, Y., Langer, M., Yu, F., Lee, M., Liu, J., Shi, J., & Wang, Z. (2022). A GPU-specialized Inference Parameter Server for Large-Scale Deep Recommendation Models. *RecSys 2022 - Proceedings of the 16th ACM Conference on Recommender Systems*, 408–419. https://doi.org/10.1145/3523227.3546765
21. Wu, L., Hanafy, W., Abdelzaher, T., Irwin, D., Milzman, J., & Shenoy, P. (2025). *FailLite: Failure-Resilient Model Serving for Resource-Constrained Edge Environments*. 416–429. https://doi.org/10.1145/3772052.3772243

22. Xiong, Y., Jiang, Y., Yang, Z., Qu, L., Zhao, G., Liu, S., Zhong, D., Pinzur, B., Zhang, J., Wang, Y., Jose, J., Pourreza, H., Baxter, J., Datta, K., Ram, P., Melton, L., Chau, J., Cheng, P., Xiong, Y., & Zhou, L. (2025). SuperBench: A Proactive Validation System for Improving Reliability of Cloud AI Infrastructure. *ACM Transactions on Computer Systems*. https://doi.org/10.1145/3767334
23. Yang, B., al Mamun, M. A., Zhang, J. M., & Uddin, G. (2025). Hallucination Detection in Large Language Models with Metamorphic Relations. *Proceedings of the ACM on Software Engineering*, *2*(FSE), 425–445. https://doi.org/10.1145/3715735
24. Zhang, C., Sun, Y., Wu, M., Chen, J., Lei, J., Abdul-Mageed, M., Jin, R., Liu, A., Zhu, J., Park, S., Yao, N., & Long, B. (2024). EmbSum: Leveraging the Summarization Capabilities of Large Language Models for Content-Based Recommendations. *RecSys 2024 - Proceedings of the 18th ACM Conference on Recommender Systems*, 1010–1015. https://doi.org/10.1145/3640457.3688185
25. Zhang, L., Song, K., Lee, Y. Q., Guo, W., Wang, H., Li, Y., Guo, H., Liu, Y., Lian, D., & Chen, E. (2025). Killing Two Birds with One Stone: Unifying Retrieval and Ranking with a Single Generative Recommendation Model. *SIGIR 2025 - Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2224–2234. https://doi.org/10.1145/3726302.3730017
26. Zhu, Y., Yuan, H., Wang, S., Liu, J., Liu, W., Deng, C., Chen, H., Liu, Z., Dou, Z., & Wen, J.-R. (2026). Large Language Models for Information Retrieval: A Survey. *ACM Transactions on Information Systems*, *44*(1), 1–54. https://doi.org/10.1145/3748304